

Lab 8 – Genetic Algorithms & AI

1. Study the following paper that uses GA to solve the Knapsack problem.

http://www.micsymposium.org/mics_2004/Hristake.pdf

Make sure you understand it very well. The main goal here is to enable you to understand some published paper well and relate it to the content of the lecture covered in the class. This paper will also enable you to do some research work as would be needed in future e.g. when you join an industry.

Nothing is due for this BUT your understanding will be helpful to answer questions on GA in the Final exam. Also, your understanding of the paper will help you solve Problem #2 below.

2. You have 5000 MP3 files sitting on your computer's hard disk. Unfortunately, the hard disk has started making noises, and you decide that you had better back up the MP3s. Equally unfortunately, you can only burn CDs, not DVDs, on your computer. You need to minimize the number of CDs that you use, so you decide to design a genetic algorithm to choose which MP3s to put onto each CD in order to fill each CD as completely as possible. Design a genetic algorithm to solve the problem. You will need to consider how you would encode the inputs, which genetic operators are suitable, and how you would make the genetic algorithm deal with the fact that you have multiple CDs, not just one CD. (Note: You can use some ideas from #1 above).

ANS:

I would use a Genetic Algorithm to try many different ways to put the MP3 files into CDs. Each solution can be represented by a list, where each number tells which CD an MP3 file goes to.

For example, [1, 1, 2, 1, 3] means the first, second, and fourth MP3 files go to CD 1, the third file goes to CD 2, and the fifth file goes to CD 3.

At first, the algorithm creates many random solutions. Then it checks which solutions are better. A good solution should use fewer CDs and should not leave too much empty space. If one CD is over its capacity, that solution should get a lower score.

The better solutions are selected and used to create new solutions. Crossover combines parts of two solutions, and mutation randomly moves one MP3 file to another CD.

This process is repeated many times. After many generations, the algorithm should find a solution that uses a small number of CDs and fills them efficiently.

3. For each of the following Agents, develop a PEAS description of the task environment:

- a. Robot soccer player

ANS:

Performance:

Score goals, make good passes, help the team win, and avoid fouls.

Environment:

Soccer field, ball, teammates, opponents, referee, and goal.

Actuators:

Legs or wheels, kicking system, and movement system.

Sensors:

Camera, distance sensor, position sensor, and touch sensor.

b. Internet book-shopping Agent

ANS:

Performance:

Find the correct book, good price, good rating, and fast delivery.

Environment:

Online bookstores, websites, users, and payment system.

Actuators:

Search for books, show results, add book to cart, and place order.

Sensors:

Book price, title, reviews, availability, and user preferences.

c. Autonomous Mars rover

ANS:

Performance:

Collect useful information, avoid damage, save energy, and complete the mission.

Environment:

Mars surface, rocks, soil, hills, craters, and dust.

Actuators:

Wheels, robotic arm, drill, and communication antenna.

Sensors:

Camera, temperature sensor, distance sensor, and chemical sensor.

d. Mathematician's Theorem Proving Assistant

ANS:

Performance:

Give correct proofs, useful suggestions, and reduce errors.

Environment:

Mathematical problems, formulas, axioms, and the user.

Actuators:

Show proof steps, suggest formulas, and report errors.

Sensors:

User input, formulas, axioms, and previous proofs.

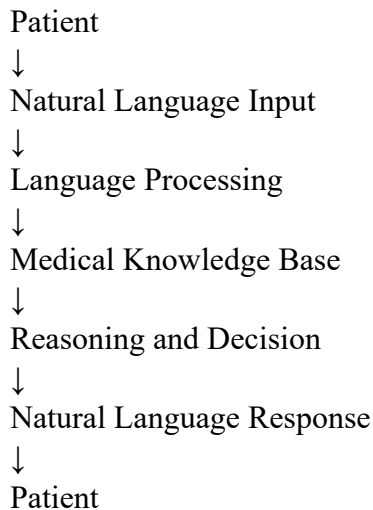
4. Consider a Natural Language based Medical Assistant Agent.

Draw its architecture along with the environment. Mention important goals and utility functions.

ANS:

A Natural Language Medical Assistant can communicate with a patient using normal language. The patient can type or speak about symptoms, medical history, or health questions.

A simple architecture is:



The environment includes the patient, doctor, hospital, medical records, lab results, and medical database.

The main goals of the agent are to give correct medical information, keep the patient safe, provide clear answers, and recognize serious situations.

A simple utility function can use accuracy, safety, relevance, and clarity. A response with high accuracy and safety should have a high utility value.

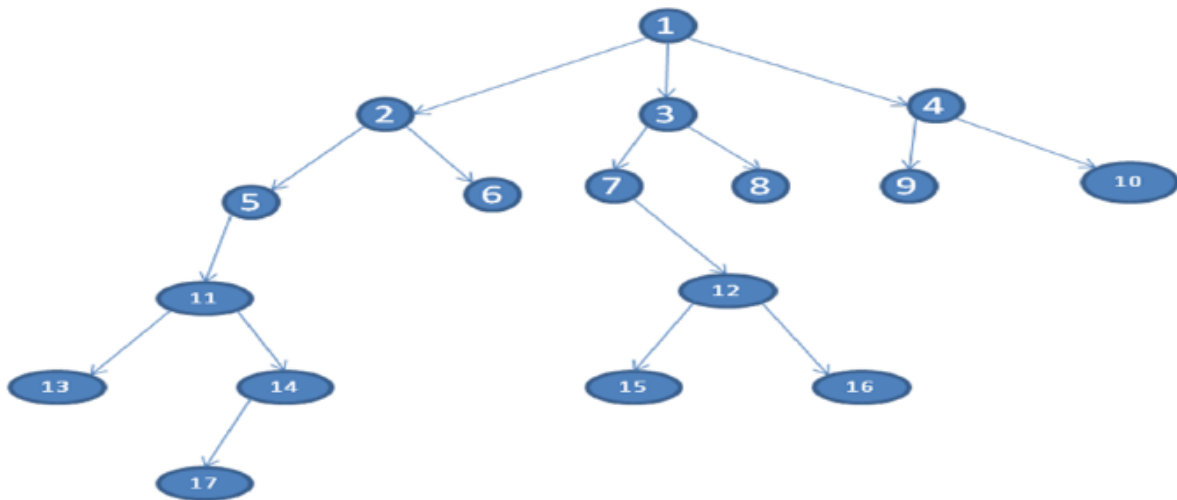
5. For the following tree, list the order in which the nodes are visited using Depth-First Search.

ANS: Depth-First Search goes as deep as possible in one branch before going back and checking another branch.

The visiting order for the tree is:

1, 2, 5, 11, 13, 14, 17, 6, 3, 7, 12, 15, 16, 8, 4, 9, 10

DFS keeps going down one path until there is no more child node. Then it goes back and continues with the next branch.



6. The travelling salesman problem (TSP) is an NP-Complete (i.e. complex – needs exponential time to solve). TSP asks the following question: "Given a list of cities and the distances between each pair of cities, what is the shortest possible route that visits each city and returns to the origin city?"

Try to solve this problem using GA. Then implement your solution in Python. You would need to import numpy, pandas, random and operator.

For the Traveling Salesman Problem, one chromosome represents one possible route through all cities.

For example:

[A, C, B, D]

This means the route is:

$A \rightarrow C \rightarrow B \rightarrow D \rightarrow A$

First, the algorithm creates many random routes. Then it calculates the total distance of every route.

A shorter route is better, so the fitness can be calculated as:

$\text{Fitness} = 1 / \text{Total Distance}$

The better routes are selected as parents. Crossover combines two parent routes to create a new route. Each city must appear only one time.

Mutation can randomly swap two cities.

For example:

Before: A B C D E

After: A D C B E

The algorithm repeats selection, crossover, mutation, and fitness calculation for many generations.

At the end, the best solution is the route with the shortest total distance.

Code in python:

```
import numpy as np
import pandas as pd
```

```
import random
import operator
```

```
class City:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def distance(self, city):
        x_distance = abs(self.x - city.x)
        y_distance = abs(self.y - city.y)

        return np.sqrt(
            x_distance ** 2 + y_distance ** 2
        )

    def __repr__(self):
        return "(" + str(self.x) + "," + str(self.y) + ")"
```

```
class Fitness:
    def __init__(self, route):
        self.route = route

    def routeDistance(self):
        total_distance = 0

        for i in range(len(self.route)):
            from_city = self.route[i]

            if i + 1 < len(self.route):
                to_city = self.route[i + 1]
            else:
                to_city = self.route[0]

            total_distance += from_city.distance(to_city)

        return total_distance

    def routeFitness(self):
        return 1 / self.routeDistance()

def createRoute(city_list):
    return random.sample(city_list, len(city_list))
```

```

def initialPopulation(pop_size, city_list):
    population = []

    for i in range(pop_size):
        population.append(createRoute(city_list))

    return population

```

```

def rankRoutes(population):
    fitness_result = {}

    for i in range(len(population)):
        fitness_result[i] = Fitness(
            population[i]
        ).routeFitness()

    return sorted(
        fitness_result.items(),
        key=operator.itemgetter(1),
        reverse=True
    )

```

```

def mutate(route, mutation_rate):
    for i in range(len(route)):

        if random.random() < mutation_rate:

            j = random.randint(
                0,
                len(route) - 1
            )

            route[i], route[j] = route[j], route[i]

    return route

```

```

# Create 10 random cities
cities = []

```

```

for i in range(10):
    cities.append(

```

```
    City(
        random.randint(0, 100),
        random.randint(0, 100)
    )
)
```

```
# Create population
population = initialPopulation(
    100,
    cities
)
```

```
# Run simple GA process
for generation in range(500):
```

```
    ranked = rankRoutes(population)
```

```
    new_population = []
```

```
    # Keep the best 20 routes
```

```
    for i in range(20):
```

```
        index = ranked[i][0]
```

```
        new_population.append(
```

```
            population[index][:]
```

```
        )
```

```
# Create more routes by mutation
```

```
while len(new_population) < 100:
```

```
    parent = random.choice(
```

```
        new_population
```

```
    )[:]
```

```
    child = mutate(
```

```
        parent,
```

```
        0.01
```

```
    )
```

```
    new_population.append(child)
```

```
population = new_population
```

```
# Get best route
best_index = rankRoutes(population)[0][0]

best_route = population[best_index]

best_distance = Fitness(
    best_route
).routeDistance()

print("Best route:")
print(best_route)

print("Distance:")
print(best_distance)
```